

Guía de verificación rápida

TFG Laboratorio de Red — ASIR

Autor: Abel

Fecha: Abril 2026

Antes de empezar — arrancar el laboratorio

Ansible despliega todo el laboratorio desde cero con un único comando. La primera ejecución tarda más porque construye las

imágenes Docker — las siguientes son mucho más rápidas al usar la caché.

```
cd ~/tfg-laboratorio-red
```

```
ansible-playbook site.yml
```

1. Redes Docker

Verificamos que las tres redes del laboratorio existen y están activas. Cada red representa una zona de seguridad distinta — WAN

(exterior), DMZ (servicios expuestos) y LAN (servicios internos).

```
docker network ls --filter driver=bridge
```

```
abel@laptop:~/tfg-laboratorio-red$ docker network ls --filter driver=bridge
NETWORK ID      NAME      DRIVER      SCOPE
c19c217fc165    bridge    bridge       local
b0c99d208996    dmz_net   bridge       local
53aebb6b2dbc     lan_net   bridge       local
c0491654ba62     wan_net   bridge       local
```

Esperado: wan_net, dmz_net, lan_net visibles.

2. Contenedores activos

Comprobamos de un vistazo que todos los contenedores están corriendo. Si alguno aparece en estado Exited o Restarting hay

un problema que investigar con `docker logs <nombre>`.

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

```
abel@laptop:~/tfg-laboratorio-red$ docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES      STATUS      PORTS
squid01    Up 3 days   3128/tcp
proxy01    Up 3 days   80/tcp, 443/tcp
web01      Up 3 days   80/tcp, 443/tcp
dhcp01     Up 3 days   67/udp
dns01      Up 3 days   53/tcp, 53/udp
ssh01      Up 3 days   22/tcp
fw01       Up 3 days
```

Esperado: fw01, ssh01, dns01, dhcp01, web01, proxy01, squid01 en estado Up.

3. Firewall — fw01

```
abel@laptop:~/tfg-laboratorio-red$ docker exec -it fw01 bash
root@e8c9d5686c35:/#
```

El firewall es el elemento central de la arquitectura — enruta el tráfico entre zonas y aplica las reglas de seguridad con `nftables`.

Verificamos que las reglas están cargadas y que la LAN puede salir a internet a través del NAT.

Ver reglas activas

docker exec fw01 nft list ruleset

```
abel@laptop:~/tfg-laboratorio-red$ docker exec -it fw01 bash
root@e8c9d5686c35:/#
root@e8c9d5686c35:/# nft list ruletest
Error: syntax error, unexpected string
list ruletest
      ^^^^^^^^
root@e8c9d5686c35:/# nft list ruleset
table inet filter {
    chain input {
        type filter hook input priority filter; policy drop;
        iif "lo" accept
        ct state established,related accept
        icmp type echo-request accept
        iif "eth1" tcp dport 22 accept
        reject with icmp admin-prohibited
    }

    chain forward {
        type filter hook forward priority filter; policy drop;
        ct state established,related accept
        iif "eth1" oif "eth0" accept
    }

    chain output {
        type filter hook output priority filter; policy accept;
    }
}
table inet nat {
    chain prerouting {
        type nat hook prerouting priority dstnat; policy accept;
    }

    chain postrouting {
        type nat hook postrouting priority srcnat; policy accept;
        oif "eth0" masquerade
    }
}
```

LAN puede salir a internet

docker exec fw01 ping -c 2 8.8.8.8

```
root@e8c9d5686c35:/# ping -c 2 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=115 time=4.64 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=115 time=4.41 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 4.411/4.527/4.643/0.116 ms
root@e8c9d5686c35:/#
```

Esperado: ruleset con chains input, forward, output y NAT masquerade. Ping responde.

4. Servidor SSH — ssh01

SSH permite administrar los contenedores de la LAN de forma remota y segura. Verificamos que el servidor escucha en el puerto 22 y acepta conexiones con la contraseña configurada.

ssh root@192.168.100.10

Contraseña: laboratorio

```
abel@laptop:~/tfg-laboratorio-red$ ssh root@192.168.100.10
root@192.168.100.10's password:
Last login: Tue Apr  7 14:43:39 2026 from 192.168.100.1
root@750590138cbd:~#
```

Esperado: acceso correcto al contenedor. hostname devuelve el ID del contenedor.

5. Servidor DNS — dns01

BIND9 resuelve los nombres del dominio laboratorio.local y reenvía las consultas externas a Google DNS y Cloudflare.

Verificamos los tres tipos de resolución: directa interna, inversa y externa.

Resolución interna

dig @192.168.100.20 ssh01.laboratorio.local +short

```
abel@laptop:~/tfg-laboratorio-red$ dig @192.168.100.20 ssh01.laboratorio.local +short
192.168.100.10
abel@laptop:~/tfg-laboratorio-red$
```

Resolución inversa

dig @192.168.100.20 -x 192.168.100.10 +short

```
abel@laptop:~/tfg-laboratorio-red$ dig @192.168.100.20 -x 192.168.100.10 +short
ssh01.laboratorio.local.
abel@laptop:~/tfg-laboratorio-red$
```

Resolución externa

dig @192.168.100.20 google.com +short

```
abel@laptop:~/tfg-laboratorio-red$ dig @192.168.100.20 www.google.com +short
142.251.153.119
142.251.157.119
142.251.151.119
142.251.154.119
142.251.152.119
142.251.150.119
142.251.155.119
142.251.156.119
abel@laptop:~/tfg-laboratorio-red$
```

Esperado: 192.168.100.10, ssh01.laboratorio.local., IP de Google.

6. Servidor DHCP — dhcp01

ISC DHCP Server asigna automáticamente IPs del rango 192.168.100.100-200 a los clientes de la LAN. Lanzamos un contenedor

Alpine sin IP fija para simular un cliente que se conecta por primera vez.

```
docker run --rm --network lan_net alpine udhcpc -i eth0 -t 5 -n
```

```
udhcpc: started, v1.37.0
ip: can't send flush request: Operation not permitted
udhcpc: broadcasting discover
udhcpc: broadcasting select for 192.168.100.101, server 192.168.100.30
udhcpc: lease of 192.168.100.101 obtained from 192.168.100.30, lease time 86400
```

Esperado: lease of 192.168.100.1xx obtained from 192.168.100.30.

7. Servidor web — web01

Nginx sirve contenido estático con HTTP y HTTPS usando un certificado SSL autofirmado. El puerto 80 redirige automáticamente a

HTTPS — requisito de seguridad básico en cualquier servidor web moderno.

HTTP redirige a HTTPS

```
curl -v http://172.21.0.20 2>&1 | grep "< HTTP"
```

HTTPS devuelve la página

```
curl -k https://172.21.0.20 | grep "<title>"
```

Esperado: código 301 en HTTP. Título Laboratorio de Red — TFG ASIR en HTTPS. También verificable desde el navegador en

https://172.21.0.20.

8. Proxy inverso — proxy01

Nginx actúa como único punto de entrada desde el exterior hacia web01. El cliente nunca habla directamente con web01 — proxy01

recibe la petición, la reenvía internamente y devuelve la respuesta, ocultando completamente el servidor real.

Acceder a web01 a través del proxy

```
curl -k https://172.21.0.10 | grep "<title>"
```

HTTP redirige a HTTPS

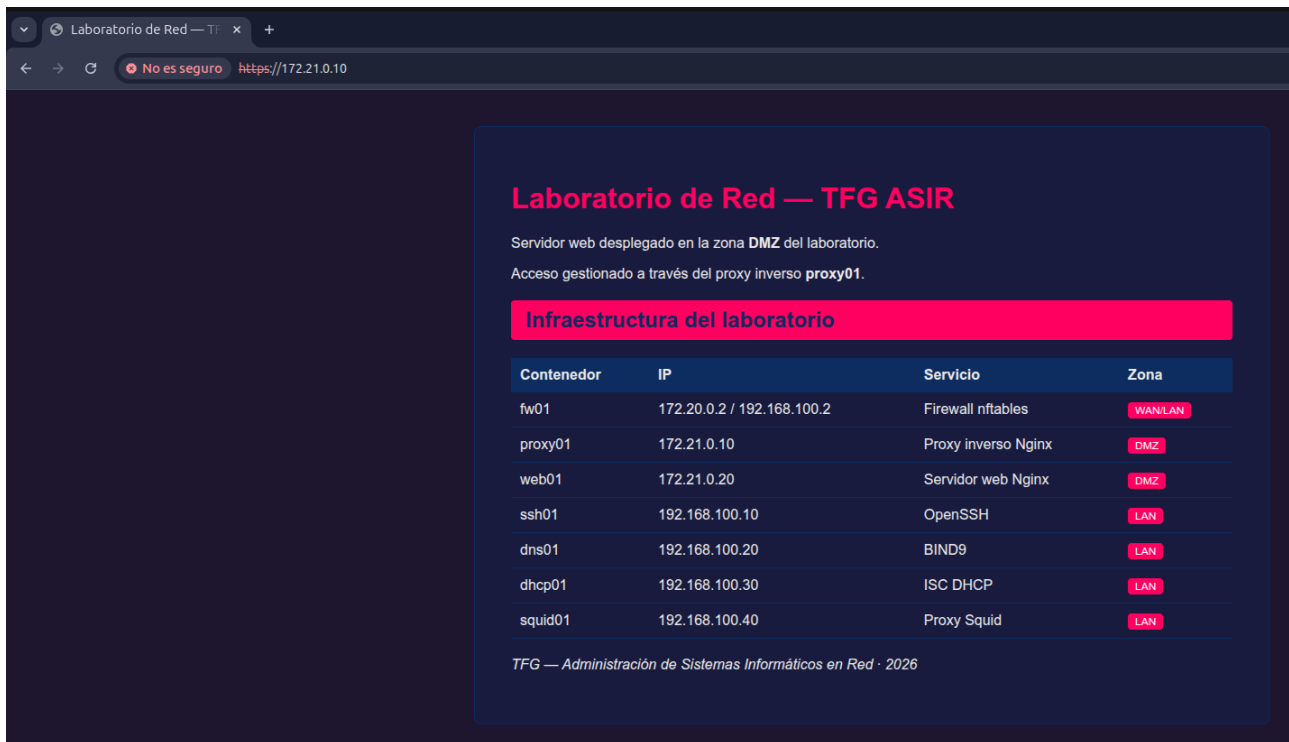
```
curl -v http://172.21.0.10 2>&1 | grep "< HTTP"
```

```
abel@laptop:~/tfg-laboratorio-red$ curl -v http://172.21.0.20 2>&1 | grep "< HTTP"
< HTTP/1.1 301 Moved Permanently
abel@laptop:~/tfg-laboratorio-red$ curl -k https://172.21.0.20 | grep "<title>"
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           %             %             Dload  Upload  Total    Spent    Left  Speed
100  2648  100  2648    0     0  146k      0  --:--:-- --:--:-- --:--:-- 152k
<title>Laboratorio de Red - TFG ASIR</title>
abel@laptop:~/tfg-laboratorio-red$
```

Esperado: misma página que web01 pero accediendo por 172.21.0.10. Código 301 en HTTP.

También verificable desde el

navegador en <https://172.21.0.10>.



9. Proxy de salida — squid01 Squid intercepta y registra todo el tráfico web saliente de los clientes de la LAN. Abrimos dos terminales — en una monitorizamos los logs en tiempo real y en la otra simulamos una petición a través del proxy para ver cómo queda registrada.

Terminal 1 — monitorizar logs

```
docker exec squid01 tail -f /var/log/squid/access.log
```

Terminal 2 — hacer una petición a través del proxy

```
curl -x 192.168.100.40:3128 http://google.com -s -o /dev/null -w "%{http_code}"
```

```
abel@laptop:~/tfg-laboratorio-red$ docker exec squid01 tail -f /var/log/squid/access.log
192.168.100.1 - - [10/Apr/2026:08:36:12 +0000] "GET http://google.com/ HTTP/1.1" 301 903 "-" "curl/8.5.0" TCP_MISS:HIER_DIRECT
```

Esperado: código 301 en Terminal 2. En Terminal 1 aparece una línea de log con TCP_MISS y la URL solicitada.

10. Verificación rápida completa

Una sola vista que resume el estado de todo el laboratorio. Útil al inicio de cada sesión de trabajo para confirmar que todo está en pie antes de continuar.

```
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"
```

```
docker network ls --filter driver=bridge
```

```
^Cabel@laptop:~/tfg-laboratorio-red$ docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"
docker network ls --filter driver=bridge


| NAMES   | IMAGE               | STATUS    | PORTS           |
|---------|---------------------|-----------|-----------------|
| squid01 | tfg/squid:latest    | Up 3 days | 3128/tcp        |
| proxy01 | tfg/proxy:latest    | Up 3 days | 80/tcp, 443/tcp |
| web01   | tfg/web:latest      | Up 3 days | 80/tcp, 443/tcp |
| dhcp01  | tfg/dhcp:latest     | Up 3 days | 67/udp          |
| dns01   | tfg/dns:latest      | Up 3 days | 53/tcp, 53/udp  |
| ssh01   | tfg/ssh:latest      | Up 3 days | 22/tcp          |
| fw01    | tfg/firewall:latest | Up 3 days |                 |



| NETWORK      | ID      | NAME   | DRIVER | SCOPE |
|--------------|---------|--------|--------|-------|
| c19c217fc165 | bridge  | bridge | local  |       |
| b0c99d208996 | dmz_net | bridge | local  |       |
| 53aebb6b2dbc | lan_net | bridge | local  |       |
| c0491654ba62 | wan_net | bridge | local  |       |


abel@laptop:~/tfg-laboratorio-red$
```